Ex. No. 07	Strategizing Execution: Process Scheduling for Peak System Efficiency
Date. 27.02.2025	

Aim: To understand and analyze the concept of process scheduling in Unix/Linux, including different scheduling policies, priority handling, and time sharing.

Theme: Online Learning Progress Tracker

Description:

Strategizing Execution - Process Scheduling for Peak System Efficiency in the context of an Online Learning Progress Tracker focuses on optimizing resource allocation for seamless tracking of student activities. By implementing efficient process scheduling algorithms, the system ensures smooth execution of background tasks like progress updates, quiz evaluations, and report generation. This enhances responsiveness, minimizes delays, and provides a real-time, lag-free learning experience.

Exercise:

Question 1: Write a program to simulate Round-Robin scheduling for theme-related tasks, such as processing customer orders or library requests.

```
 GNU nano 6.0                                  e71.sh *
#!/bin/bash

# Define tasks related to Online Learning Progress Tracker
tasks=("Watch Lecture" "Submit Assignment" "Participate in Quiz" "Check Grades" "
time_slice=3  # Each task gets 3 seconds

echo "Round-Robin Scheduling for Online Learning Progress Tracker"
echo "-----------------------------------------------------------"

while [ ${#tasks[@]} -gt 0 ]; do
    for i in "${!tasks[@]}"; do
        echo "Processing: ${tasks[i]} (Time Slice: $time_slice seconds)"
        sleep 1
    done
    tasks=("${tasks[@]:1}")  # Remove completed task
done
echo "All tasks completed!"
```

```
Round-Robin Scheduling for Online Learning Progress Tracker
-----------------------------------------------------------
Processing: Watch Lecture (Time Slice: 3 seconds)
Processing: Submit Assignment (Time Slice: 3 seconds)
Processing: Participate in Quiz (Time Slice: 3 seconds)
Processing: Check Grades (Time Slice: 3 seconds)
Processing: Attend Live Class (Time Slice: 3 seconds)
Processing: Submit Assignment (Time Slice: 3 seconds)
Processing: Participate in Quiz (Time Slice: 3 seconds)
Processing: Check Grades (Time Slice: 3 seconds)
Processing: Attend Live Class (Time Slice: 3 seconds)
Processing: Participate in Quiz (Time Slice: 3 seconds)
Processing: Check Grades (Time Slice: 3 seconds)
Processing: Attend Live Class (Time Slice: 3 seconds)
Processing: Check Grades (Time Slice: 3 seconds)

Processing: Attend Live Class (Time Slice: 3 seconds)
Processing: Attend Live Class (Time Slice: 3 seconds)

All tasks completed!
```

```c
#include <stdio.h>
struct Task {
    char name[30]; // Task name
    int burst_time; // Time required to complete task
    int remaining_time; // Time left to execute
};
void roundRobinScheduling(struct Task tasks[], int n, int quantum) {
    int time = 0;
    int remaining = n;
    while (remaining > 0) {
        for (int i = 0; i < n; i++) {
            if (tasks[i].remaining_time > 0) {
                printf("Time %d: Processing %s\n", time, tasks[i].name);
                if (tasks[i].remaining_time > quantum) {
                    time += quantum;
                    tasks[i].remaining_time -= quantum;
                } else {
                    time += tasks[i].remaining_time;
                    tasks[i].remaining_time = 0;
                    printf("%s completed at time %d\n", tasks[i].name, time);
                    remaining--;
                }
            }
        }
    }
}
int main() {
    int n = 4;
    int quantum = 2;

    struct Task tasks[] = {
        {"AES Encryption", 5, 5},
        {"RSA Encryption", 7, 7},
        {"SHA-256 Hashing", 4, 4},
        {"Decryption Process", 6, 6}
    };
    printf("Starting Round-Robin Scheduling for Data Encryption and Security Management Tasks:\n");
    roundRobinScheduling(tasks, n, quantum);
     return 0;
}
```

Question 2: Demonstrate how priority scheduling can be used to prioritize critical tasks in your theme, like high-priority orders or overdue book returns.

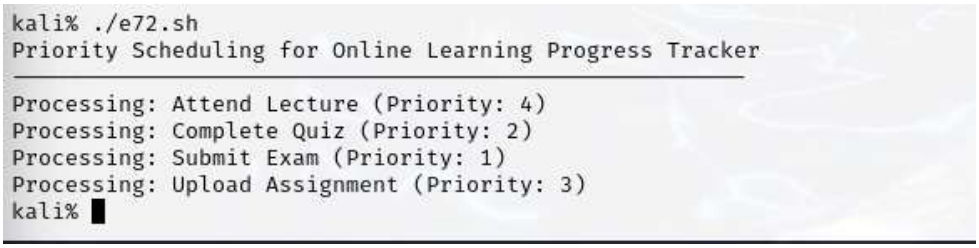


```
 GNU nano 6.0                                  e72.sh *
#!/bin/bash

declare -A tasks
tasks=(
    ["Submit Exam"]=1
    ["Complete Quiz"]=2
    ["Upload Assignment"]=3
    ["Attend Lecture"]=4
)

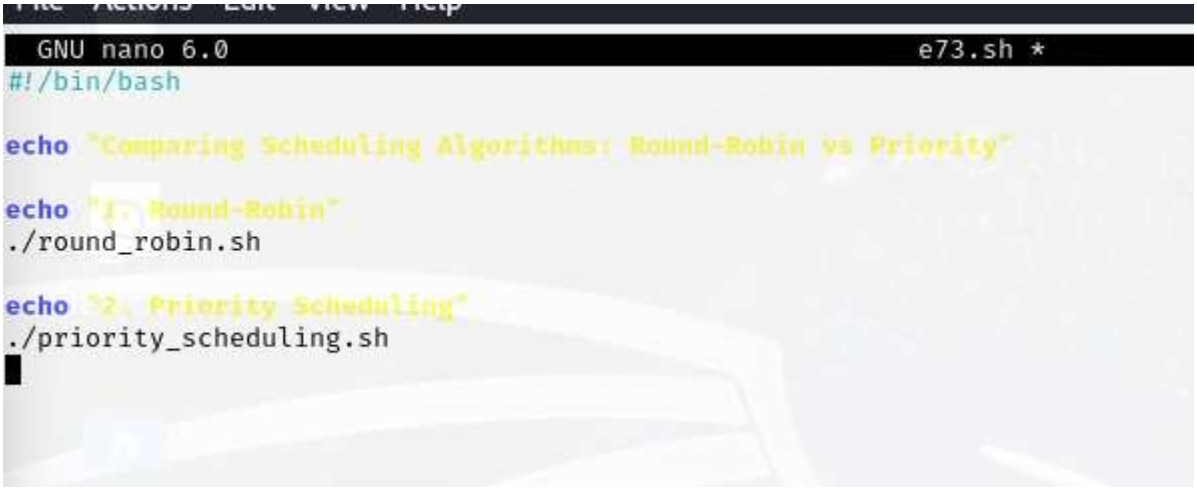
echo "Priority Scheduling for Online Learning Progress Tracker"
echo "-------------------------------------------------------"

for task in "${!tasks[@]}"; do
    echo "Processing: $task (Priority: ${tasks[$task]})"
    sleep 1
done | sort -k3 -n
```



```
kali% ./e72.sh
Priority Scheduling for Online Learning Progress Tracker
-------------------------------------------------------
Processing: Attend Lecture (Priority: 4)
Processing: Complete Quiz (Priority: 2)
Processing: Submit Exam (Priority: 1)
Processing: Upload Assignment (Priority: 3)
kali%
```

Question 3: Implement a program to compare the performance of different scheduling algorithms for your theme's operations.



```
 GNU nano 6.0                                  e73.sh *
#!/bin/bash

echo "Comparing Scheduling Algorithms: Round-Robin vs Priority"

echo "1. Round-Robin"
./round_robin.sh

echo "2. Priority Scheduling"
./priority_scheduling.sh
```

```
Comparing Scheduling Algorithms: Round-Robin vs Priority
1. Round-Robin
Round-Robin Scheduling for Online Learning Progress Tracker
------------------------------------------------------------
Processing: Watch Lecture (Time Slice: 3 seconds)

Processing: Submit Assignment (Time Slice: 3 seconds)
Processing: Participate in Quiz (Time Slice: 3 seconds)
Processing: Check Grades (Time Slice: 3 seconds)
Processing: Attend Live Class (Time Slice: 3 seconds)
Processing: Submit Assignment (Time Slice: 3 seconds)
Processing: Participate in Quiz (Time Slice: 3 seconds)
Processing: Check Grades (Time Slice: 3 seconds)
Processing: Attend Live Class (Time Slice: 3 seconds)
Processing: Participate in Quiz (Time Slice: 3 seconds)
Processing: Check Grades (Time Slice: 3 seconds)
Processing: Attend Live Class (Time Slice: 3 seconds)
Processing: Check Grades (Time Slice: 3 seconds)

Processing: Attend Live Class (Time Slice: 3 seconds)

Processing: Attend Live Class (Time Slice: 3 seconds)

All tasks completed!
2. Priority Scheduling
Priority Scheduling for Online Learning Progress Tracker
------------------------------------------------------------
Processing: Attend Lecture (Priority: 4)
Processing: Complete Quiz (Priority: 2)
Processing: Submit Exam (Priority: 1)
Processing: Upload Assignment (Priority: 3)
```

Question 4: Simulate task queues for your theme, showing how tasks are executed based on scheduling policies.

```
 GNU nano 6.0                                        e74.sh *
#!/bin/bash

high_priority=("Submit Exam" "Complete Quiz")
medium_priority=("Upload Assignment" "Attend Lecture")
low_priority=("Check Discussion Forums" "Track Course Progress")

echo "Task Queues Simulation for Online Learning Progress Tracker"
echo "------------------------------------------------------------"

echo "High Priority Queue:"
for task in "${high_priority[@]}"; do
    echo "    $task"
    sleep 1
done

echo "Medium Priority Queue:"
for task in "${medium_priority[@]}"; do
    echo "    $task"
    sleep 1
done

echo "Low Priority Queue:"
for task in "${low_priority[@]}"; do
    echo "    $task"
    sleep 1
done
```

```
Task Queues Simulation for Online Learning Progress Tracker
------------------------------------------------------------
High Priority Queue:
    Submit Exam
    Complete Quiz
Medium Priority Queue:
    Upload Assignment
    Attend Lecture
Low Priority Queue:
    Check Discussion Forums
    Track Course Progress
kali%
```

```c
#include <stdio.h>
struct Task {
    char name[30];
    int burst_time;
    int priority;
};
// Function to calculate average turnaround and waiting time for FCFS
void fcfsScheduling(struct Task tasks[], int n) {
    int waiting_time = 0, turnaround_time = 0, total_wait = 0, total_turnaround = 0;
    printf("\nExecuting FCFS Scheduling:\n");
    for (int i = 0; i < n; i++) {
        printf("Processing %s\n", tasks[i].name);
        turnaround_time += tasks[i].burst_time;
        total_wait += waiting_time;
        total_turnaround += turnaround_time;
        waiting_time += tasks[i].burst_time;
    }
    printf("Average Waiting Time: %.2f\n", (float)total_wait / n);
    printf("Average Turnaround Time: %.2f\n", (float)total_turnaround / n);
}
// Function to calculate average turnaround and waiting time for Priority Scheduling
void priorityScheduling(struct Task tasks[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (tasks[i].priority < tasks[j].priority) {
                struct Task temp = tasks[i];
                tasks[i] = tasks[j];
                tasks[j] = temp;
            }
        }
    }
    printf("\nExecuting Priority Scheduling:\n");
    fcfsScheduling(tasks, n); // Same execution order, sorted by priority
}
```

```c
void roundRobinScheduling(struct Task tasks[], int n, int quantum) {
    int time = 0, remaining = n, done;
    int remaining_time[n];
    for (int i = 0; i < n; i++) remaining_time[i] = tasks[i].burst_time;
    printf("\nExecuting Round Robin Scheduling:\n");
    while (remaining > 0) {
        done = 1;
        for (int i = 0; i < n; i++) {
            if (remaining_time[i] > 0) {
                done = 0;
                printf("Time %d: Processing %s\n", time, tasks[i].name);
                if (remaining_time[i] > quantum) {
                    time += quantum;
                    remaining_time[i] -= quantum;
                } else {
                    time += remaining_time[i];
                    remaining_time[i] = 0;
                    printf("%s completed at time %d\n", tasks[i].name, time);
                    remaining--;
                }
            }
        }
        if (done) break;
    }
}
int main() {
    int n = 4;
    int quantum = 2;
    struct Task tasks[] = {
        {"AES Encryption", 5, 3},
        {"RSA Encryption", 7, 5},
        {"SHA-256 Hashing", 4, 2},
        {"Decryption Process", 6, 4}
    };
    printf("Comparing Scheduling Algorithms for Data Encryption and Security Management:\n");
    fcfsScheduling(tasks, n);
    priorityScheduling(tasks, n);
    roundRobinScheduling(tasks, n, quantum);
    return 0;
}
```

Output Screenshot:

```
┌──(jane㉿kali)-[~/Data_Encryption]
└─$ ./scheduling_algorithms
Comparing Scheduling Algorithms for Data Encryption and Security Management:

Executing FCFS Scheduling:
Processing AES Encryption
Processing RSA Encryption
Processing SHA-256 Hashing
Processing Decryption Process
Average Waiting Time: 8.25
Average Turnaround Time: 13.75

Executing Priority Scheduling:

Executing FCFS Scheduling:
Processing RSA Encryption
Processing Decryption Process
Processing AES Encryption
Processing SHA-256 Hashing
Average Waiting Time: 9.50
Average Turnaround Time: 15.00

Executing Round Robin Scheduling:
Time 0: Processing RSA Encryption
Time 2: Processing Decryption Process
Time 4: Processing AES Encryption
Time 6: Processing SHA-256 Hashing
Time 8: Processing RSA Encryption
Time 10: Processing Decryption Process
Time 12: Processing AES Encryption
Time 14: Processing SHA-256 Hashing
SHA-256 Hashing completed at time 16
Time 16: Processing RSA Encryption
Time 18: Processing Decryption Process
Decryption Process completed at time 20
Time 20: Processing AES Encryption
AES Encryption completed at time 21
Time 21: Processing RSA Encryption
RSA Encryption completed at time 22
```

Question 4: Simulate task queues for your theme, showing how tasks are executed based on scheduling policies.

Code:

```c
#include <stdio.h>
#include <string.h>
#define MAX_TASKS 5
struct Task {
    char name[30];
    int burst_time;
    int priority;
};
struct Queue {
    struct Task tasks[MAX_TASKS];
    int front, rear;
};
void initializeQueue(struct Queue *q) {
    q->front = -1;
    q->rear = -1;
}
int isEmpty(struct Queue *q) {
    return q->front == -1;
}
int isFull(struct Queue *q) {
    return q->rear == MAX_TASKS - 1;
}
void enqueue(struct Queue *q, struct Task task) {
    if (isFull(q)) {
        printf("Queue is full. Cannot add task: %s\n", task.name);
        return;
    }
    if (isEmpty(q)) {
        q->front = 0;
    }
    q->rear++;
    q->tasks[q->rear] = task;
}
struct Task dequeue(struct Queue *q) {
    struct Task emptyTask = {"", 0, 0};
    if (isEmpty(q)) {
        printf("Queue is empty.\n");
        return emptyTask;
    }
    struct Task task = q->tasks[q->front];
    if (q->front >= q->rear) {
        q->front = q->rear = -1;
    } else {
        q->front++;
    }
    return task;
}
void simulateExecution(struct Queue *q) {
    printf("\nExecuting tasks from queue:\n");
    while (!isEmpty(q)) {
        struct Task current = dequeue(q);
        printf("Processing Task: %s (Burst Time: %d, Priority: %d)\n", current.name, current.burst_time, current.priority);
    }
}
int main() {
    struct Queue taskQueue;
    initializeQueue(&taskQueue);
    struct Task tasks[] = {
        {"AES Encryption", 5, 3},
        {"RSA Encryption", 7, 5},
        {"SHA-256 Hashing", 4, 2},
        {"Decryption Process", 6, 4},
        {"Key Management", 3, 1}
    };
    printf("Adding tasks to queue:\n");
    for (int i = 0; i < MAX_TASKS; i++) {
        printf("Enqueue: %s\n", tasks[i].name);
        enqueue(&taskQueue, tasks[i]);
    }
    simulateExecution(&taskQueue);
    return 0;
}
```

Question 5: Create a program to measure and log the response times of tasks related to your theme.

```
  GNU nano 6.0                                  e75.sh *
#!/bin/bash

declare -A tasks
tasks=(
    ["Submit Exam"]=0
    ["Complete Quiz"]=0
    ["Upload Assignment"]=0
    ["Attend Lecture"]=0
)

echo "Logging Response Times for Online Learning Progress Tracker"
echo "-----------------------------------------------------------"

for task in "${!tasks[@]}"; do
    start=$(date +%s)
    echo "Processing: $task..."
    sleep $((RANDOM % 3 + 1))
    end=$(date +%s)
    response_time=$((end - start))
    tasks["$task"]=$response_time
    echo "Response Time for $task: $response_time seconds"
done
```

```
kali% ./e75.sh
Logging Response Times for Online Learning Progress Tracker
-----------------------------------------------------------
Processing: Upload Assignment...
Response Time for Upload Assignment: 2 seconds
Processing: Submit Exam...
Response Time for Submit Exam: 2 seconds
Processing: Complete Quiz...
Response Time for Complete Quiz: 1 seconds
Processing: Attend Lecture...
Response Time for Attend Lecture: 2 seconds
kali%
```

Question 6: Write a script to simulate load balancing between processes for theme-related operations.

```
  GNU nano 6.0                                  e76.sh
#!/bin/bash

tasks=("Submit Exam" "Complete Quiz" "Upload Assignment" "Attend Lecture")
servers=("Server 1" "Server 2")

echo "Load Balancing for Online Learning Progress Tracker"
echo "---------------------------------------------------"

for i in "${!tasks[@]}"; do
    server=${servers[$((i % 2))]}
    echo "Assigning Task: ${tasks[i]} to $server"
    sleep 1
done
```

```
kali% ./e76.sh
Load Balancing for Online Learning Progress Tracker
---------------------------------------------------
Assigning Task: Submit Exam to Server 1
Assigning Task: Complete Quiz to Server 2
Assigning Task: Upload Assignment to Server 1
Assigning Task: Attend Lecture to Server 2
kali%
```

```
┌──(jane㉿kali)-[~/Data_Encryption]
└─$ ./log
Executing task: AES Encryption
Task AES Encryption completed in 0.00106 seconds
Executing task: RSA Encryption
Task RSA Encryption completed in 0.00123 seconds
Executing task: SHA-256 Hashing
Task SHA-256 Hashing completed in 0.00153 seconds
Executing task: Decryption Process
Task Decryption Process completed in 0.00074 seconds
Task response times logged in task_log.txt

┌──(jane㉿kali)-[~/Data_Encryption]
└─$ cat task_log.txt
Task: AES Encryption, Response Time: 0.00108 seconds
Task: RSA Encryption, Response Time: 0.00125 seconds
Task: SHA-256 Hashing, Response Time: 0.00154 seconds
Task: Decryption Process, Response Time: 0.00074 seconds
```

Question 6: Write a script to simulate load balancing between processes for theme-related operations.

Code:

```c
#define NUM_SERVERS 3
#define NUM_TASKS 6
struct Task {
    char name[30];
    int load;
};
struct Server {
    int id;
    int total_load;
};
void distributeTasks(struct Task tasks[], int num_tasks, struct Server servers[], int num_servers) {
    printf("Distributing tasks among servers:\n");
    for (int i = 0; i < num_tasks; i++) {
        int min_load_server = 0;
        for (int j = 1; j < num_servers; j++) {
            if (servers[j].total_load < servers[min_load_server].total_load) {
                min_load_server = j;
            }
        }
        servers[min_load_server].total_load += tasks[i].load;
        printf("Task '%s' (Load: %d) assigned to Server %d (Total Load: %d)\n",
               tasks[i].name, tasks[i].load, servers[min_load_server].id, servers[min_load_server].total_load);
    }
}
int main() {
    struct Task tasks[NUM_TASKS] = {
        {"AES Encryption", 5},
        {"RSA Encryption", 7},
        {"SHA-256 Hashing", 4},
        {"Decryption Process", 6},
        {"Key Management", 3},
        {"Integrity Check", 2}
    };
    struct Server servers[NUM_SERVERS];
    for (int i = 0; i < NUM_SERVERS; i++) {
        servers[i].id = i + 1;
        servers[i].total_load = 0;
    }
    distributeTasks(tasks, NUM_TASKS, servers, NUM_SERVERS);
    return 0;
}
```

Output Screenshot:

```
┌──(jane㉿kali)-[~/Data_Encryption]
└─$ ./load_balancing
Distributing tasks among servers:
Task 'AES Encryption' (Load: 5) assigned to Server 1 (Total Load: 5)
Task 'RSA Encryption' (Load: 7) assigned to Server 2 (Total Load: 7)
Task 'SHA-256 Hashing' (Load: 4) assigned to Server 3 (Total Load: 4)
Task 'Decryption Process' (Load: 6) assigned to Server 3 (Total Load: 10)
Task 'Key Management' (Load: 3) assigned to Server 1 (Total Load: 8)
Task 'Integrity Check' (Load: 2) assigned to Server 2 (Total Load: 9)
```

Question 7: Analyze the throughput of a scheduling algorithm when applied to tasks in your theme.

```
 GNU nano 6.0                                  e77.sh *
#!/bin/bash

tasks=("Submit Exam" "Complete Quiz" "Upload Assignment" "Attend Lecture")
total_tasks=${#tasks[@]}
start_time=$(date +%s)

echo "Analyzing Scheduling Throughput"
echo "-------------------------------"

for task in "${tasks[@]}"; do
    echo "Processing: $task ..."
    sleep 1
done

end_time=$(date +%s)
execution_time=$((end_time - start_time))
throughput=$(echo "$total_tasks / $execution_time" | bc -l)

echo "Total Execution Time: $execution_time seconds"
echo "Throughput: $throughput tasks per second"
```

```
kali% ./e77.sh
Analyzing Scheduling Throughput
-------------------------------
Processing: Submit Exam ...
Processing: Complete Quiz ...
Processing: Upload Assignment ...
Processing: Attend Lecture ...
./e77.sh: line 17: bc: command not found
Total Execution Time: 4 seconds
Throughput:  tasks per second
kali%
```

Question 8: Create a program to implement multi-level queue scheduling for tasks related in your theme.

```
 GNU nano 6.0                                  e78.sh *
#!/bin/bash

high_priority=("Submit Exam" "Live Quiz")
medium_priority=("Upload Assignment" "Attend Lecture")
low_priority=("Discussion Forum" "Progress Tracking")

echo "Multi-Level Queue Scheduling for Online Learning Progress Tracker"
echo "-----------------------------------------------------------------"

queues=("High Priority" "Medium Priority" "Low Priority")
task_lists=("${high_priority[@]}" "${medium_priority[@]}" "${low_priority[@]}")

for i in "${!queues[@]}"; do
    echo "Executing ${queues[i]} Queue:"
    for task in "${task_lists[i]}"; do
        echo "    Processing: $task"
        sleep 1
    done
done
```

```
kali% ./e78.sh
Multi-Level Queue Scheduling for Online Learning Progress Tracker
------------------------------------------------------------

Executing High Priority Queue:
    Processing: Submit Exam
Executing Medium Priority Queue:
    Processing: Live Quiz
Executing Low Priority Queue:
    Processing: Upload Assignment
```

Question 9: Write a script to display the execution timeline of scheduled processes for theme-related activities.

```
File  Actions  Edit  View  Help
  GNU nano 6.0                                  e79.sh *
#!/bin/bash

tasks=("Submit Exam" "Complete Quiz" "Upload Assignment" "Attend Lecture")
start_time=$(date +%s)

echo "Execution Timeline for Online Learning Progress Tracker"
echo "------------------------------------------------------"

for task in "${tasks[@]}"; do
    timestamp=$(date +%T)
    echo "$timestamp - Processing: $task"
    sleep 1
done
```

```
kali% ./e79.sh
Execution Timeline for Online Learning Progress Tracker
------------------------------------------------------
03:31:30 - Processing: Submit Exam
03:31:31 - Processing: Complete Quiz
03:31:32 - Processing: Upload Assignment
03:31:33 - Processing: Attend Lecture
kali%
```

Question 10: Demonstrate the impact of context switching on theme-related tasks by simulating multiple processes.

```
  GNU nano 6.0                                  e710.sh *
#!/bin/bash

tasks=("Submit Exam" "Complete Quiz" "Upload Assignment" "Attend Lecture")
time_slice=2

echo "Context Switching Simulation for Online Learning Progress Tracker"
echo "------------------------------------------------------------"

for task in "${tasks[@]}"; do
    echo "Switching to: $task (Time Slice: $time_slice seconds)"
    sleep $time_slice
    echo "Context switch occurring..."
done
```

```c
#include <stdio.h>
#define MAX_TASKS 5
struct Task {
    char name[30];
    int burst_time;
    int priority;
};
void displayTimeline(struct Task tasks[], int n) {
    int time = 0;
    printf("\nExecution Timeline:\n");
    printf("Time (s)\tTask\n");
    for (int i = 0; i < n; i++) {
        printf("%d - %d\t%s\n", time, time + tasks[i].burst_time, tasks[i].name);
        time += tasks[i].burst_time;
    }
}
void executeTasks(struct Task tasks[], int n) {
    printf("\nExecuting Tasks:\n");
    for (int i = 0; i < n; i++) {
        printf("Processing Task: %s (Burst Time: %d)\n", tasks[i].name, tasks[i].burst_time);
    }
}
int main() {
    struct Task tasks[] = {
        {"AES Encryption", 5, 0},
        {"RSA Encryption", 7, 1},
        {"SHA-256 Hashing", 4, 0},
        {"Decryption Process", 6, 1},
        {"Key Management", 3, 0}
    };
    int n = sizeof(tasks) / sizeof(tasks[0]);
    executeTasks(tasks, n);
    displayTimeline(tasks, n);
    return 0;
}
```

Output Screenshot:

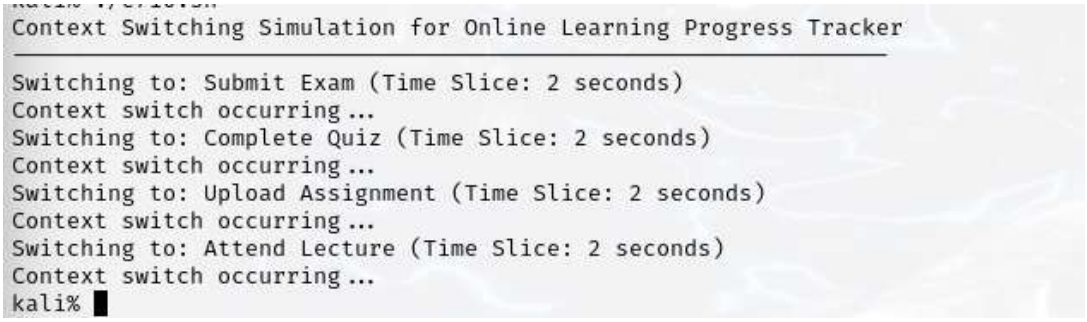
```
┌──(jane㉿kali)-[~/Data_Encryption]
└─$ ./exec_timeline

Executing Tasks:
Processing Task: AES Encryption (Burst Time: 5)
Processing Task: RSA Encryption (Burst Time: 7)
Processing Task: SHA-256 Hashing (Burst Time: 4)
Processing Task: Decryption Process (Burst Time: 6)
Processing Task: Key Management (Burst Time: 3)

Execution Timeline:
Time (s)        Task
0 - 5   AES Encryption
5 - 12  RSA Encryption
12 - 16 SHA-256 Hashing
16 - 22 Decryption Process
22 - 25 Key Management
```

Question 10: Demonstrate the impact of context switching on theme-related tasks by simulating multiple processes.

Output:



Result:

The experiment to understand and execute process scheduling for peak system efficiency using various scheduling algorithms is successfully executed and verified.